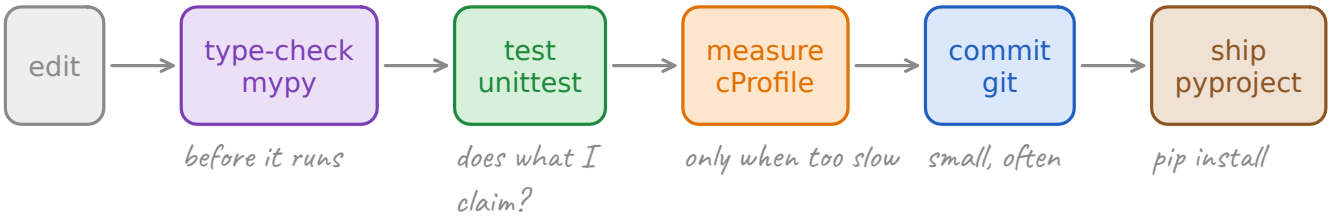
Chapter 17

Quality: tests, speed, types, shipping

how professionals know their code works, and keep it working

Writing code that works once is the easy half. Quality means being able to **show** it works, find out **why** it's slow, catch mistakes **before** running it, let other people **install** it, and **undo** any change. Each of those has a standard Python tool, and every other language has the same five:

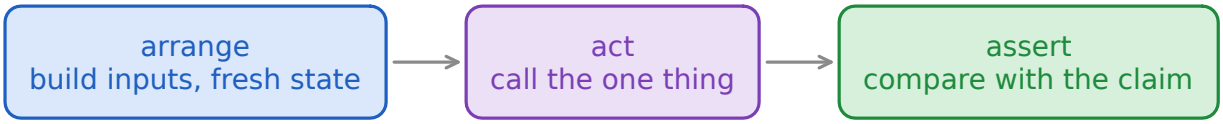


the question	Python tool	in the stdlib?
Does it do what I claim, and still after my change?	`unittest`, `doctest`, `unittest.mock`	yes
Where does the time go?	`timeit`, `cProfile` + `pstats`	yes
Do the types line up, before I run anything?	type hints + `mypy` or `pyright`	hints yes, checker no
Can someone else install and run it?	`pyproject.toml`, `venv`, `pip`, `build`	venv yes, rest pip
What changed, when, why, and can I undo it?	Git	separate program

The habit matters more than the tool: a test written before the fix, a measurement taken before the optimisation, a commit made before the experiment. The practice repo you've been using is built exactly this way: every exercise is a unittest suite.

unittest: claims the computer checks

A test is a **claim about behaviour** written as code: "a book returned 4 days late costs 1.00". Each test follows the same three steps:




```
1  import sys, unittest
2  from datetime import date, timedelta
3
4  def late_fee(due, returned, per_day=0.25, cap=5.0):
5      days = (returned - due).days
6      return 0.0 if days <= 0 else min(days * per_day, cap)
7
8  class LateFeeTest(unittest.TestCase):
9      due = date(2026, 3, 1)
10
11     def test_on_time_is_free(self):
12         self.assertEqual(late_fee(self.due, self.due), 0.0)
13
14     def test_per_day(self):
15         self.assertEqual(late_fee(self.due, date(2026, 3, 5)), 1.0)
16
17     def test_capped(self):
18         self.assertEqual(late_fee(self.due, date(2026, 6, 1)), 5.0)
19
20     def test_rejects_text(self):
21         with self.assertRaises(TypeError):
22             late_fee(self.due, "2026-03-05")
23
24 def run(*cases, v=1):                    # what `python -m unittest` does for you
25     load = unittest.defaultTestLoader.loadTestsFromTestCase
26     suite = unittest.TestSuite(load(c) for c in cases)
27     unittest.TextTestRunner(stream=sys.stdout, verbosity=v).run(suite)
28
29 run(LateFeeTest, v=2)
```

```
test_capped (__main__.LateFeeTest.test_capped) ... ok
test_on_time_is_free (__main__.LateFeeTest.test_on_time_is_free) ... ok
test_per_day (__main__.LateFeeTest.test_per_day) ... ok
test_rejects_text (__main__.LateFeeTest.test_rejects_text) ... ok

----------------------------------------------------------------------
Ran 4 tests in 0.000s

OK
```

line 8: subclass TestCase

line 11: every test_* method is one test

line 21: the error is the expected behaviour

A failing test tells you which claim broke and shows both values. Here the claim is wrong, not the code: floats can't hold 0.1 exactly (Ch 11), so compare money with a tolerance or use `Decimal`:

```
class TenPence(unittest.TestCase):
    def test_three_days(self):
        self.assertEqual(late_fee(date(2026, 3, 1), date(2026, 3, 4), per_day=0.1), 0.3)

run(TenPence)
```

```
F
======================================================================
FAIL: test_three_days (__main__.TenPence.test_three_days)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "<chapter 17>", line 3, in test_three_days
AssertionError: 0.30000000000000004 != 0.3

----------------------------------------------------------------------
Ran 1 test in 0.000s

FAILED (failures=1)
```

assert method	checks	use it for
`assertEqual(a, b)`	`a == b`, with a diff on failure	most things; lists and dicts get a diff
`assertAlmostEqual(a, b)`	equal to 7 decimal places	floats
`assertTrue(x)` / `assertIn(a, b)`	truthiness / membership	flags, containers
`assertRaises(E)` as a `with`	the block raises `E`	error paths (they need tests too)
`self.subTest(**info)`	labels one case in a loop	table-driven tests (next)

Which cases? Partitions and boundaries

You can't test every input, so split the inputs into **partitions** that should behave the same, test one from each, and add the **boundaries** between them, where off-by-one bugs live:

partition	days late	expected fee	why this case
returned early	`-3`	`0.0`	negative days must not give a refund
on the due date	`0`	`0.0`	boundary: `<` vs `<=`
first late day	`1`	`0.25`	boundary: the first charge
just under the cap	`19`	`4.75`	boundary below the cap
exactly at the cap	`20`	`5.0`	boundary: `min` kicks in
far over the cap	`90`	`5.0`	the cap holds

```
class Boundaries(unittest.TestCase):
    def test_table(self):
        due = date(2026, 3, 1)
        for days, fee in [(-3, 0.0), (0, 0.0), (1, 0.25), (19, 4.75), (20, 5.0), (90, 5.0)]:
            with self.subTest(days=days):
                self.assertEqual(late_fee(due, due + timedelta(days)), fee)

run(Boundaries)
```

```
.
----------------------------------------------------------------------
Ran 1 test in 0.000s

OK
```

line 5: a failure names the case and keeps going

setUp and addCleanup: a fresh world for every test

```
import sqlite3

class LoanStore(unittest.TestCase):
    def setUp(self):                                # runs before every test
        self.db = sqlite3.connect(":memory:")
        self.addCleanup(self.db.close)              # runs after, even if the test fails
        self.db.execute("CREATE TABLE loans (title TEXT, due TEXT)")

    def test_insert(self):
        self.db.execute("INSERT INTO loans VALUES ('Dune', '2026-03-01')")
        self.assertEqual(self.db.execute("SELECT count(*) FROM loans").fetchone(), (1,))

    def test_starts_empty(self):                    # test_insert's row is gone
        self.assertEqual(self.db.execute("SELECT count(*) FROM loans").fetchone(), (0,))

run(LoanStore, v=2)
```

```
test_insert (__main__.LoanStore.test_insert) ... ok
test_starts_empty (__main__.LoanStore.test_starts_empty) ... ok

----------------------------------------------------------------------
Ran 2 tests in 0.000s

OK
```

RULE OF THUMB

Tests must not depend on each other or on their order. Each one builds what it needs (in `setUp`, or a temporary folder from `tempfile`) and leaves nothing behind. If a test passes alone but fails in the suite, some state is leaking.

doctest: examples that can't go stale

A `>>>` example in a docstring documents the function **and** is a test: `doctest` runs it and compares the printed result. Good for small pure functions; use unittest for anything with setup.

```
1  import doctest
2
3  def fine(days_late, per_day=0.25, cap=5.0):
4      '''Late fee for a book, capped.
5
6      >>> fine(0)
7      0.0
8      >>> fine(4)
9      1.0
10     >>> fine(100)
11     5.0
12     '''
13     return min(max(days_late, 0) * per_day, cap)
14
15 runner = doctest.DocTestRunner()
16 for test in doctest.DocTestFinder().find(fine, globs={"fine": fine}):
17     runner.run(test)
18 print(runner.summarize(verbose=False))
```

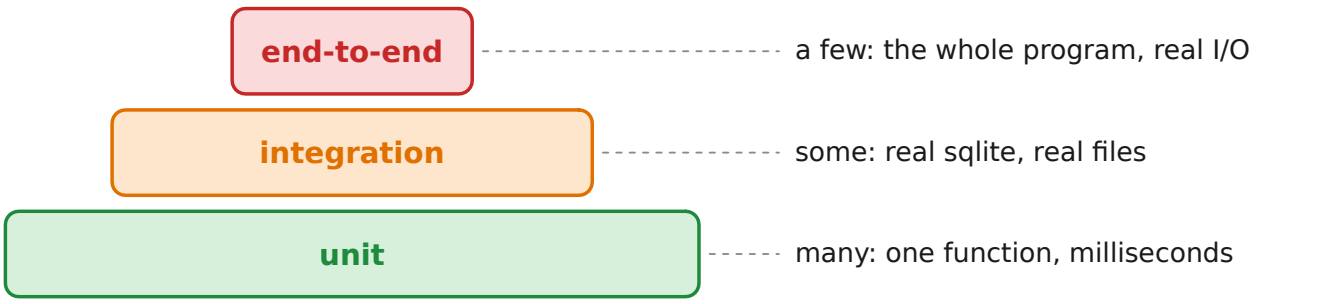
```
TestResults(failed=0, attempted=3)
```

line 6: the REPL session, pasted in

line 16: python -m doctest file.py does this

How big should a test be?

A **unit** test checks one function on its own; an **integration** test checks pieces working together, with a real database or real files; an **end-to-end** test runs the whole program the way a user would. Aim for this shape:



slower, costlier, flakier as you go up

unittest.mock: replacing the outside world

Some code talks to things a test can't use: a mail server, the network, the clock. A **test double** stands in for them. The cleanest seam is a parameter, so the test can pass a `Mock`, which accepts any call and records it:

```
1  from unittest.mock import Mock
2
3  def send_reminders(loans, today, mailer):
4      for who, title, due in loans:
5          if due < today:
6              mailer.send(who, f"'{title}' was due {due:%d %b}")
7
8  mailer = Mock()
9  loans = [("ana@x.org", "Dune", date(2026, 3, 1)), ("ben@x.org", "Emma", date(2026, 4, 1))]
10 send_reminders(loans, date(2026, 3, 10), mailer)
11 mailer.send.assert_called_once_with("ana@x.org", "'Dune' was due 01 Mar")
12 print(mailer.send.call_count, mailer.send.call_args)
```

```
1 call('ana@x.org', "'Dune' was due 01 Mar")
```

line 3: the dependency comes in as an argument

line 8: no SMTP server needed

line 11: raises AssertionError if not

double	what it does	in Python
dummy	fills a parameter, never used	`None`, `object()`
stub	returns canned answers	`Mock(return_value=...)`
fake	a real, simpler implementation	sqlite `:memory:`, a dict, `io.StringIO`
spy / mock	records calls so you can check them	`Mock()` + `assert_called_with`

patch: when you can't pass the dependency in

`patch` swaps a name for a `MagicMock` during a `with` block and puts the original back afterwards. `return_value` decides what the fake returns; `side_effect` makes it raise, so you can test the error path without unplugging the network:

```
1  import io, json, urllib.request
2  from unittest.mock import patch
3
4  def book_title(isbn):
5      url = f"https://openlibrary.org/isbn/{isbn}.json"
6      with urllib.request.urlopen(url, timeout=5) as r:
7          return json.load(r)["title"]
8
9  with patch("urllib.request.urlopen") as urlopen:
10     urlopen.return_value.__enter__.return_value = io.BytesIO(b'{"title": "Dune"}')
11     print(book_title("9780441013593"), "|", urlopen.call_args.args[0])
12
13 with patch("urllib.request.urlopen", side_effect=TimeoutError("too slow")):
14     try:
15         book_title("9780441013593")
16     except TimeoutError as e:
17         print("error path tested:", e)
```

```
Dune | https://openlibrary.org/isbn/9780441013593.json
error path tested: too slow
```

line 10: what `with ... as r` will give

line 13: the fake raises instead

TRAP

Patch the name **where it is looked up**, not where it was defined. If `loans.py` did `from urllib.request import urlopen`, it holds its own reference, so you must patch `"loans.urlopen"`. Patching `urllib.request.urlopen` would leave that copy untouched.

WHY

Mock too much and your tests check **how** the code works (which calls it makes) instead of **what** it achieves, so every refactor breaks them. Prefer fakes (an in-memory database, a temporary folder) and keep mocks for the edges of your program: network, mail, clock, randomness.

Measure, then optimise

Programmers are famously bad at guessing where time goes. The professional loop never skips a step:




```python
import cProfile, pstats

def is_member(members, mid):
    return mid in members

def report(n):
    members = [f"m{i:05}" for i in range(n)]
    loans = [f"m{i * 7 % (2 * n):05}" for i in range(n)]
    return sum(is_member(members, m) for m in loans)

with cProfile.Profile() as prof:
    report(4000)
funcs = pstats.Stats(prof).get_stats_profile().func_profiles
for name, p in sorted(funcs.items(), key=lambda kv: -kv[1].tottime)[:3]:
    print(f"{name[:12]:12} calls={p.ncalls:>5}  own time={p.tottime:.3f} s")
```

```
is_member    calls= 4000  own time=0.065 s
report       calls=    1  own time=0.002 s
<genexpr>    calls= 4001  own time=0.001 s
```

line 4: `in` on a list: a linear scan

line 11: record every call

line 14: sort by time spent inside each function

`is_member` takes nearly all the time, and it's called 4000 times, each scanning up to 4000 names. Change one line, membership in a `set` (Ch 10), then check it's faster **and** still gives the same answer:

```python
import timeit

def report_fast(n):
    members = {f"m{i:05}" for i in range(n)}      # a set: the only change
    loans = [f"m{i * 7 % (2 * n):05}" for i in range(n)]
    return sum(is_member(members, m) for m in loans)

assert report(4000) == report_fast(4000)
for fn in (report, report_fast):
    t = min(timeit.repeat(lambda: fn(4000), number=3, repeat=3)) / 3
    print(f"{fn.__name__:12} {t * 1000:7.1f} ms")
```

```
report         65.5 ms
report_fast     1.7 ms
```

line 8: same result first

line 10: min of repeats: least noise

HONEST NOTE

Timings on this page were measured while building the book, so yours will differ. From the shell: `python3 -m timeit -s "xs = list(range(1000))" "999 in xs"` for one-liners, and `python3 -m cProfile -s tottime script.py` for a whole program. Profilers slow code down, so trust their proportions more than their seconds.

Type hints, and a checker that reads them

Python checks types only at run time, and only when an operation fails. **Type hints** write your intent down; Python itself ignores them:

```
def double(n: int) -> int:
    return n * 2

print(double(21), double("ha"), double.__annotations__)
```

```
42 haha {'n': <class 'int'>, 'return': <class 'int'>}
```

← runs fine: hints aren't enforced

A separate **static checker**, `mypy` or `pyright` (both pip installs; editors run them as you type), reads the hints and finds the mismatch without running anything:

loans.py

```
from dataclasses import dataclass

@dataclass
class Loan:
    title: str
    days: int

def double(n: int) -> int:
    return n * 2
double("ha")

def find(loans: list[Loan], title: str) -> Loan | None:
    return next((x for x in loans if x.title == title), None)
print(find([], "Dune").title)
```

```
$ mypy loans.py
loans.py:10: error: Argument 1 to "double" has incompatible type "str"; expected "int"  [arg-type]
loans.py:14: error: Item "None" of "Loan | None" has no attribute "title"  [union-attr]
Found 2 errors in 1 file (checked 1 source file)
```

The second error is the valuable kind: `find` can return `None`, and the caller forgot. That bug would pass every test that happens to find the book, then crash in production.

write	means
`list[str]`, `dict[str, int]`, `tuple[int, ...]`	containers and what they hold
`Loan | None`	maybe missing: the checker makes you handle `None`
`Callable[[int], str]` (from `collections.abc`)	a function taking an int, returning a str
`Literal["due", "title"]`	only these exact values
`def first[T](xs: list[T]) -> T:` (3.12+)	generic: the output type follows the input
`class Mailer(Protocol): def send(...)`	anything with a matching `send` method fits

```
1  from typing import Protocol
2
3  class Mailer(Protocol):
4      def send(self, to: str, text: str) -> None: ...
5
6  class PrintMailer:                              # never mentions Mailer
7      def send(self, to: str, text: str) -> None:
8          print(f"to {to}: {text}")
9
10 def remind(m: Mailer) -> None:
11     m.send("ana@x.org", "Dune is due")
12
13 def first[T](items: list[T], default: T) -> T:
14     return items[0] if items else default
15
16 remind(PrintMailer())
17 print(first([3, 1], 0), first([], "none"))
```

```
to ana@x.org: Dune is due
3 none
```

line 3: structural: any class with this send fits

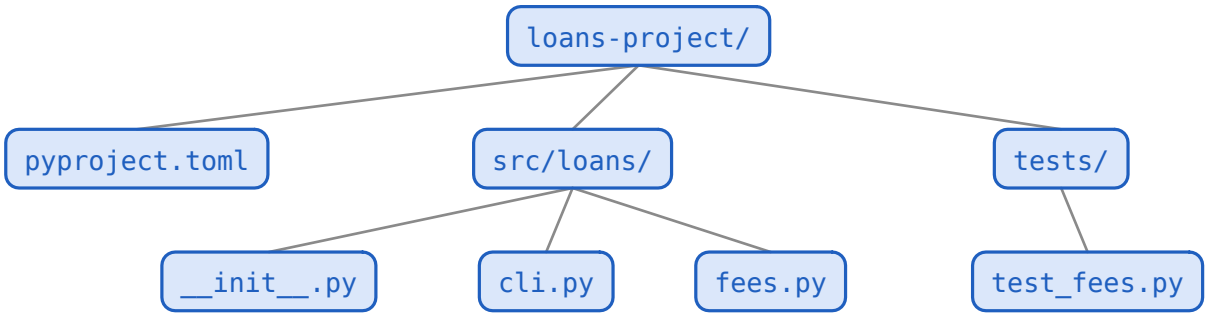
line 11: T: the checker knows first(...) is an int

HONEST NOTE

Hints are **gradual**: unannotated code is simply not checked, so you can add them one module at a time. They don't make Python faster. Since 3.14 annotations are evaluated lazily (PEP 649), so a hint can name a class defined further down the file.

pyproject.toml: making it installable

A project others can install has a standard shape. The **src layout** keeps your package out of the current folder, so tests run against the **installed** code, exactly as users will (Ch 2's sys.path):




```
1  import tomllib
2
3  PYPROJECT = """
4  [build-system]
5  requires = ["setuptools>=77"]
6  build-backend = "setuptools.build_meta"
7
8  [project]
9  name = "loans"
10 version = "0.3.0"
11 requires-python = ">=3.12"
12 dependencies = []
13
14 [project.optional-dependencies]
15 dev = ["mypy"]
16
17 [project.scripts]
18 loans = "loans.cli:main"
19 """
20 cfg = tomllib.loads(PYPROJECT)                # the stdlib reads TOML (it can't write it)
21 print(cfg["project"]["name"], cfg["project"]["version"], cfg["project"]["scripts"])
```

```
loans 0.3.0 {'loans': 'loans.cli:main'}
```

line 5: the tool that builds the package

line 12: runtime needs (Ch 19)

line 15: extras: pip install -e '.[dev]'

line 18: a command → your main(argv=None)

from a fresh clone to a running command and a wheel

```
$ python3 -m venv .venv && source .venv/bin/activate
(.venv) $ pip install -e ".[dev]"          # editable: edits in src/ apply at once
(.venv) $ loans --help                     # the [project.scripts] entry point
usage: loans [-h] [-v] {add,list} ...
(.venv) $ python -m unittest discover -s tests
....
----------------------------------------------------------------------
Ran 4 tests in 0.002s

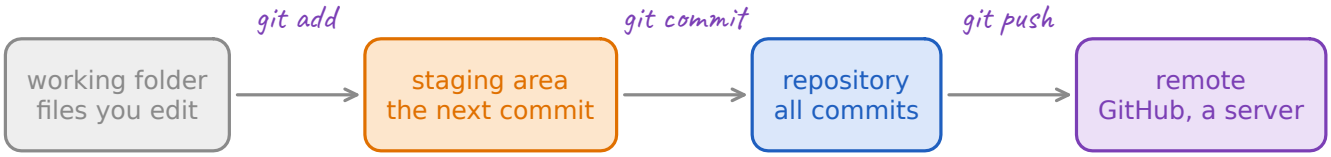
OK
(.venv) $ python -m build                  # pip install build; output goes to dist/
Successfully built loans-0.3.0.tar.gz and loans-0.3.0-py3-none-any.whl
```

WHY

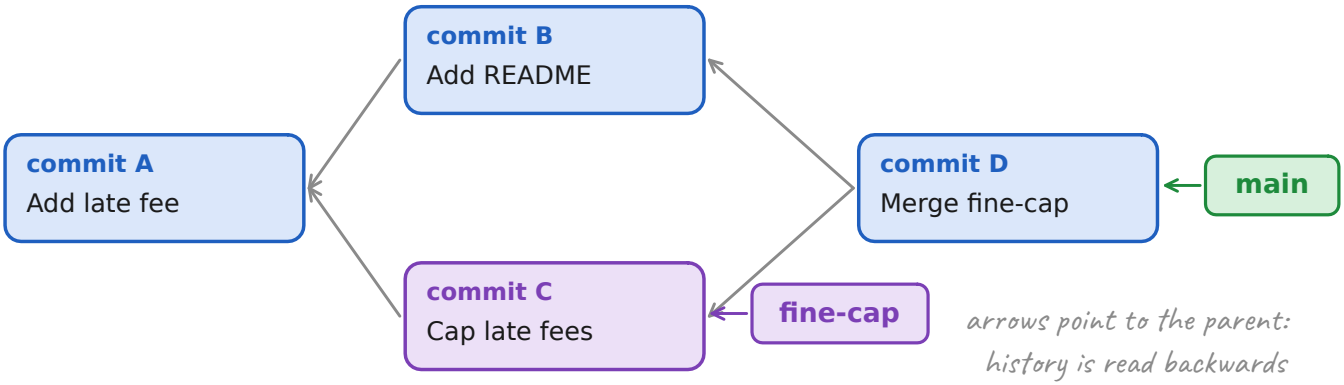
A **wheel** (`.whl`) is a zip of your package plus metadata; `pip install` just unpacks it into `site-packages`. `pyproject.toml` replaced `setup.py` so that any build tool (setuptools, hatchling, flit, uv) can read the same file. Which dependencies to trust is Ch 19.

Git: every change, recorded and undoable

Git stores **snapshots** of your whole project, called commits. Each commit records its parent(s), its author and a message. A change travels from your folder into the repository, then out to a shared copy:



A **branch** is only a movable name pointing at one commit, so branching is instant. Do an experiment on a branch, then merge it: the merge commit has two parents.



a real session, run while building this page

```
# write fees.py
$ git init -q -b main
$ git add fees.py
$ git commit -qm "Add late fee"
$ git switch -qc fine-cap
# edit fees.py: cap the fee at 5.00
$ git commit -qam "Cap late fees at 5.00"
$ git switch -q main
# write README.md
$ git add README.md
$ git commit -qm "Add README"
$ git merge -q --no-edit fine-cap
$ git log --oneline --graph
*   ab8cc0c Merge branch 'fine-cap'
|\
| * 308497a Cap late fees at 5.00
* | f0d8ad4 Add README
|/
* 55c9eb9 Add late fee
```

everyday command	does
`git status`, `git diff`	what changed since the last commit (check before every commit)
`git add -p`	stage piece by piece: one commit = one logical change
`git switch -c name` / `git merge name`	start an experiment / bring it back
`git restore file`	throw away uncommitted edits to a file
`git revert <commit>`	a new commit that undoes an old one (safe on shared history)

TRAP

A secret committed once stays in the history even after you delete the file, and anyone who clones gets it. Add `.env`, keys and `*.db` to `.gitignore` **before** the first commit. If a secret was ever pushed, **rotate it**: rewriting history doesn't remove copies people already have.

▢ Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 17.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 17.1` shows other valid solutions and their trade-offs.

17.1 build from a spec Tests that catch bugs (the checker plants them)
17.2 fix the bug Tests that pass alone but fail together
17.3 refactor, tests stay green Make the reminder job testable
17.4 measure and explain Profile it, then make it fast
17.5 learn an unseen module Documentation that tests itself
17.6 learn an unseen module Read type hints at run time
17.7 mini-project Scaffold an installable project

PASSPORT STAMP ask the same question of every new language (Appendix B)

3. Typing

Dynamic and strong at run time. Optional hints (`list[str]`, `X | None`, `Protocol`, generics) are ignored by Python and checked statically by `mypy` or `pyright`.

9. Modules and build

`pyproject.toml` declares the package, dependencies and commands; `venv` + `pip install -e .` for development; wheels for distribution; src layout.

12. Stdlib and tooling

`unittest` (+ `subTest`, `setUp`, `addCleanup`), `doctest`, `unittest.mock`, `timeit`, `cProfile` + `pstats`, `tomllib`, `typing`. Outside: Git, mypy, build.

IN OTHER LANGUAGES

Java: JUnit + Mockito, Maven/Gradle (`pom.xml`), static types checked by the compiler. Rust: `#[test]` and `cargo test`, `cargo bench`, `Cargo.toml`. C: no standard test framework, `make`/CMake, `perf`/`gprof`. TypeScript's types work exactly like Python's hints: checked by `tsc`, erased at run time. Git is the same in every language.

DEEP DIVE (your vault)

Vault notes: 12.3 Testing is executable claim checking · 12.4 Test selection by partitions and boundaries · 12.7 Test doubles and mocking · 12.9 Property-based and metamorphic testing · 12.10 Coverage and mutation testing · 12.18 Type hints · 12.19 Static analysis with mypy · 14.15 Microbenchmarking · 14.16 Profiling · 12.13 Git's object model · 12.14 Branching, merging, and conflicts · 12.21 Continuous integration.